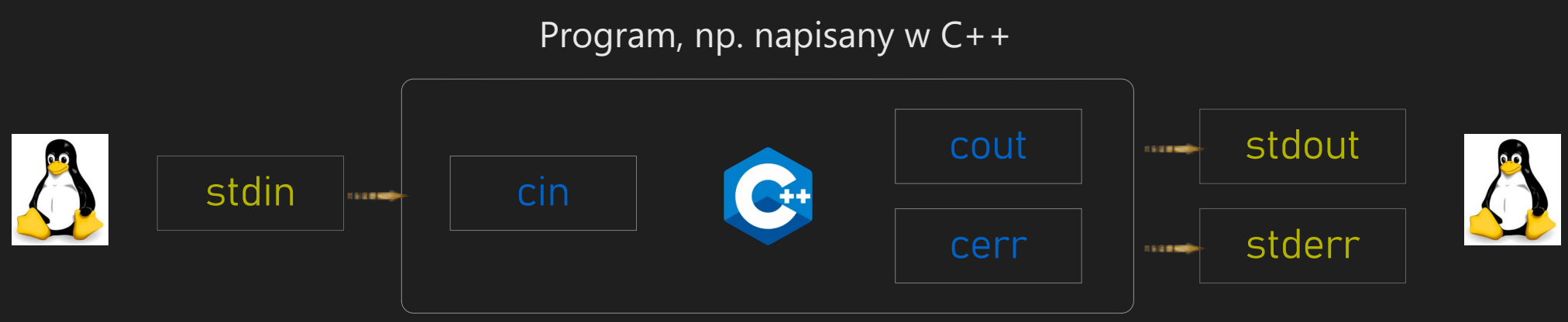
Narzędzia Wspierające Programowanie

Bash – dalsze rozszerzenie

W tym skrypcie omówimy szereg dalszych aspektów Basha:
- strumienie i przetwarzanie potokowe
- zmienne
- pętle i warunki
- skrypty.

Ciekawe kompendium znajdziemy np. [tutaj].

Strumienie



Gdy program w Linuxie wypisuje tekst do terminala, to tak naprawdę przekazuje tekst do tzw. "standardowego strumienia wyjścia".

W C++ polecenie `cout` przekazuje tekst do linuxowego strumienia stdout,
a komenda `cerr` - do strumienia stderr (jest to strumień błędów) .

Z kolei, do programu Linux podłącza standardowy strumień wejścia stdin:
Gdy za pomocą `cin` użytkownik podaje tekst z klawiatury, tekst przechodzi przez ten strumień.

- **Strumienie wyjścia:** `stdout` i `stderr`

Tekst w `stdout` można przekierować
z ekranu na plik, używając znaku `>`

```
$ ls > myout
```

Jeśli plik istniał wcześniej, to zostanie nadpisany.
Ale można poprzez `>>` dopisać do końca pliku:

```
$ echo "End of list" >> myout
```

Co z komunikatami o błędach: jak przekierować `stderr` na plik?
Okazuje się, że powyższy linuksowy strumień "zwykły" ma nr 1,
a strumień błędów - nr 2. Trzeba więc napisać tak:

```
$ ls * i.dont.exist 1>myout 2>myerr
```

a jeśli chcemy, aby komunikaty o błędach trafiały
jednak do `stdout`, to piszemy tak:

```
$ ls * i.dont.exist 1>myout 2>&1
```

Można też przekierować strumień donikąd.
Służy temu miejsce docelowe `/dev/null`. Np:

```
$ ls * i.dont.exist 1>/dev/null
```

- **Strumień wejścia:** `stdin`

Jak dotąd, użytkownik zapytany – odpowiadał z klawiatury.
Ale można pod `stdin` podpiąć plik, używając `<` .
Jako przykład, użyjmy polecenie `wc -w`,
które poda liczbę słów w badanym tekście.

```
$ wc -w < myout
```

Z kolei, używając `<<<` , podamy do `stdin`
jedną linię tekstu. Jeżeli w linii są jawnie spacje,
to trzeba ją otoczyć przez `''` lub `" "` . Dla przykładu,
wczytajmy do kalkulatora `bc` działania:

```
$ bc -l <<< '2 / 3'
0.6666666666666
```

Dodatek

Można też, używając operatora `<<` , podać tekst
ad hoc, umawiając się na hasło końca (np. EOF):

```
$ wc -w << EOF
a b c
d e f
EOF
```

→ odpowiedź: 6

⦿ Dostrzegamy, że Linux ma dedykowane narzędzia do konkretnych zadań. Przyjrzyjmy się kilku narzędziom, które wkrótce zaprzęgniemy do tzw. “przetwarzania potokowego”

- `wc {plik} lub <strumień` (word count) : narzędzie do zliczania w tekście. Opcje, np: Opcją wskazujemy, co ma zliczać. Np.:

  `-l` podaj liczbę linii
  `-w` podaj liczbę słów
  `-m` podaj liczbę znaków

  Np.: `$ wc -l myout`

- `cut {opcja} {plik} lub <strumień` Wycina z tekstu kawałek i wypisuje go. Opcje:

  `-c {zakres}` przedział znaków, akceptuj w zakresie { }
  `-f {zakres}` słowa (fields = pola), akceptuj w zakresie { }. Domyślny delimiter: Tab
  `-f {zakres} -d {znak}` Domyślny delimiter: {znak}
  `--complement` odwróć regułę filtrowania

  `{zakres}` : W jaki sposób możemy go zapisać? np. 2 2- -5 2-5

  Np.:

```
$ cut -c 6- <<< 'I am just a nice line.'                  → just a nice line.
$ cut -c -4 <<< 'I am just a nice line.'                  → I am
$ cut -c 6-9 <<< 'I am just a nice line.'                 → just
$ cut -f 5 -d ' ' <<< 'I am just a nice line.'            → nice
$ cut -f 5 -d ' ' --complement <<< 'I am just a nice line.' → I am just a line.
```

- **tr** `{z} {na} {plik} lub <strumień` Przekształca tekst, wypisując go. Warianty:

  `-d 'zbiór'` skasuje znaki ze zbioru (z koszyka znaków)
  `-s 'znak'` skasuje bezpośrednie powtórzenia znaku

  Np.:

```
$ tr a b <<< 'I am just a nice line.'              →  I bm just b nice line.
$ tr 'a-f' 'A-F' <<< 'I am just a nice line.'      →  I Am just A niCE linE.
$ tr '1-4' 'a-d' <<< 7654321                       →  765dcba
$ tr -d '1-4' <<< 7654321                          →  765
$ tr -s ' ' <<< '| One  Two   Tree |'              →  | One Two Tree |
```

Uwaga: jeśli do tr podamy plik, to tr nie zmieni jego wnętrza. Jedynie wypisze przeprocesowany tekst. Aby zmienić frazę w środku pliku:

```
sed -i 's/stare_slowo/nowe_slowo/g' {mojPlik}
```

⦿ Plik z kolumnami danych

W naszym katalogu dostępny jest plik `people1.dat` (uwaga: delimiterem kolumn jest tu tab)
Wykonajmy kilka manipulacji na tych danych:

`$ head -2 people1.dat` wypisze tylko 2 pierwsze linie pliku

`$ tail -2 people1.dat` wypisze tylko 2 ostatnie linie

`$ cut -f 2 --complement people1.dat` usunie 2. kolumnę (domyślny delimiter = tab)

- `sort {plik} lub <strumien` sortuje linie pliku. Opcje:

  `-r` w odwróconym porządku
  `-k {nr kol.}` względem danej kolumny

  Np. `$ sort -k 2 -r people1.dat`

⊕ *Dodatek*

- `paste {plik1} {plik2}` zestawia dwa pliki w sąsiednie kolumny.

  Np. rozdzielmy dane na 2 pliki: `names.txt` i `values.txt`, a następnie scalmy obok siebie.

```
$ cut -f -2 people1.dat > names.txt
$ cut -f 3- people1.dat < values.txt

$ paste names.txt values.txt
```

⦿ Przetwarzanie potokowe

- Poznaliśmy różne komendy pracujące potokowo (`stdin` → komenda → `stdout/stderr`).
  Linux ma operator `|` (pipe), który treść w `stdout` wstawia do `stdin` nowego polecenia.
  Przykłady:

  `$ ls -1 | wc -l` Poznamy, ile plików/ścieżek ma katalog

  `$ cat people1.dat | sort -k 3 | cut -f 1` Ukaże imiona, posortowane po 3. kolumnie

  `$ cat people1.dat | grep 2 | grep 3 | wc -l` Zliczy linie mające znak 2 oraz 3.

  Oczywiście, na końcu wynik można zapisać do pliku, znakiem `>` .

⦿ Przetwarzanie potokowe (c. d.)

- Istnieją polecenia, które nie czytają ze `stdin`, tylko z opcji wywołania. Choćby `echo`.
  Aby `stdout` z poprzednika podać do opcji wywołania następnika, korzysta się z `xargs`. Np.:

```
$ ls | xargs echo
```

Uwaga: `xargs` działa osobno na każdej linii `stdout` poprzednika. Zajdzie więc różnica:

```
$ ls -1 | wc -l
$ ls -1 | xargs wc -l
```

z listy plików, zostanie podana jej długość
dla każdego pliku, `wc` poda jego długość

Bywa jednak, że chcemy poprzedni `stdout` umieścić w innym miejscu następnego polecenia. Do tego służy opcja `-I {symbol}`. W ten sposób uczymy Linux, że gdy napotka taki symbol, to ma w jego miejsce wkleić treść linii ze `stdout` od poprzednika.

Dla przykładu, utwórzmy wpierw 3 pliki:

```
$ touch a1.txt a2.txt a3.txt
```

Teraz zmienimy im rozszerzenia, ale zrobimy to seryjnie:

```
$ ls -1 a*.txt | cut -d '.' -f 1 | xargs -I {}  mv -v {}.txt {}.dat
```

- Skoro mówimy o operacjach na kolejnych liniach wejścia, to można też wyszukiwać pliki (łącznie z podścieżkami) i na nich wykonywać nasze operacje:

```
$ find . -name {wzor_pliku} -exec {komenda} {}  \;
```

↳ na każdym znalezionym pliku o danym wzorze wykona komendę, podstawiając pod {} nazwę pliku

Np.

```
$ find . -name "*.C" -exec ls -og {} \; | cut -d ' ' -f 4-
```

```
$ find . -name {wzor_pliku} -execdir {komenda} {}  \;
```

↳ na każdym znalezionym pliku o danym wzorze, przejdzie do jego ścieżki i wykona komendę, podstawiając pod {} nazwę pliku.

⊕ *Dodatek*

Istnieje też polecenie `tee {plik}`, które `stdout` z poprzednika bocznikuje do pliku. Np.:

```
$ cat people1.dat | sort -k 3 | tee sorted.txt | cut -f 3
```

pośredni text → `sorted.txt`

⦿ Globbing

Wkrótce przejdziemy do pętli zakresowych w Bashu. Są to pętle m. in. po tablicach stringów czy tablicach nazw plików. Omówmy najpierw potrzebną tu cegiełkę.
Globbing = ekspandowanie zakresu stringów oraz nazw (istniejących) plików.

‣ Nawiasy klamrowe { } służą do ekspandowania stringu w tablicę, według wzorca. Np:

```
$ echo a{1..5}b                    → a1b a2b a3b a4b a5b
$ echo name{06..12..3}.dat         → name06.dat name09.dat name12.dat
$ echo name{1,7,9}.dat             → name1.dat name7.dat name9.dat
```

Stosowanie nawiasów jest bardzo pomocne przy zmienianiu nazwy pliku:

```
$ mv abc.{dat,txt}                 podmiana rozszerzenia
$ mv {,prefix_}name.out            wstawienie prefixu
```

⊕ Dodatek: powyższe nawiasy nie znają ułamków dziesiętnych. Potrafi to polecenie `seq` :

```
$ seq 1 5                  → 1 2 3 4 5            (liczby jedna pod drugą)
$ seq 1 3 10               → 1 4 7 10                (j.w.)
$ seq 1.25 0.5 2.75        → 1,25 1,75 2,25 2,75     (j.w.)
```

Nb. możemy nie lubić przecinka jako separatora ułamków. Sposób obejścia:

```
$ LC_NUMERIC=en_US seq 1.25 0.5 2.75   → 1.25 1.75 2.25 2.75
```

Więcej o nawiasach { } [tutaj]

‣ Do dalszego kroku wytwórzmy serię plików (będą puste).

```
$ touch {a..d}{1..4}.txt
$ ls ??.txt
```

‣ nawiasy kwadratowe [ ] . Ekspandują nazwy istniejących plików (nie po-prostu stringi). Np.:

```
$ ls a[13]*                        zaakceptuje pliki mające tu w nazwie 1 lub 3
$ ls b[1-3]*                       zaakceptuje pliki mające tu w nazwie od 1 do 3
$ ls [ad][2-4]*
$ ls [!ac]*                        odrzuci pliki mające tu w nazwie litery: a lub c
```

⊕ Dodatek: więcej o nawiasach [ ] [tutaj] i [tutaj]

‣ Niech K będzie komendą, która wypisze tekst do `stdout` .
Istnieją operatory podstawienia: `` `K` `` oraz `$(K)` , które wkleją ten tekst w dane miejsce. Np.:

```
$ echo Poczatek `seq 1 3` koniec      → Poczatek 1 2 3 Koniec
$ echo Poczatek $(seq 1 3) koniec     → Poczatek 1 2 3 Koniec
```

⦿ Zmienne (Variables)

Bash oferuje zmienne. Są one bez typów. Mogą mieścić dowolną kombinację znaków.
Jeśli zawierają tylko liczby, to mogą być traktowane jako liczby.
Jeśli w środku występują spacje, to treść trzeba objąć w ' ' lub " ".

```
$ var="123 abc"
$ echo $var
```

Aby użyć zmiennej, poprzedzamy ją $

Można wstawić zmienną do środka napisu. Ale trzeba „zabezpieczyć" jej nazwę, poprzez ${var}.

```
$ num=1234
$ filename=prefix_${num}.dat
$ echo $filename
```

→ prefix_1234.dat

W kombinacji z operatorem przypisania, zmienna może przechować wypis z jakiejś komendy:

```
$ mydate=`date`
$ myfiles=$(ls)
```

Prosta arytmetyka. Stosując ((.....)) na zmiennych z liczbami, można wykonać proste działania:

```
$ a=-3; b=2
$ suma=$((a + b))    ; echo $suma
$ roznica=$((a - b)) ; echo $roznica
$ iloczyn=$((a * b)) ; echo $iloczyn
$ iloraz=$((a / b))  ; echo $iloraz
$ reszta=$((a % b))  ; echo $reszta

$ (( a += 1 , b++ ))
$ echo $((a += b))
```

← Bash działa w domenie całkowitej

Aby wczytać do zmiennej (np. z klawiatury), używamy komendy read. Za opcją -p można dać monit:

```
$ read -p "Wpisz cos: " var
```

Można tej komendy użyć również do czytania ze strumienia lub z pliku. Np.:

```
$ read var <<< 'To tylko przyklad.'
$ echo $var

$ echo "To jest linia tekstu." > linia.txt
$ read var < linia.txt
$ echo $var
```

Zasięg zmiennych

Domyślnie, zmienne utworzone w naszej sesji – istnieją tylko w niej.
W Bashu tę klasę zmiennych nazywa się "shell variables" (zmienne powłoki).
Tych zmiennych nie widzą: ani sesja nadrzędna, ani w terminalu obok, ani podprocesy naszej sesji.

Ale można zmienną uwidocznić dla naszych podprocesów. Robimy to poleceniem `export` :

`$ export filename`	Udostępni tę zmienną podprocesom
`$ export var2=test`	Utworzy `var2` i udostępni ją podprocesom

Tę klasę zmiennych nazywa się "environment variables" (zmienne środowiskowe).

Dodatek: Linux oferuje predefiniowane zmienne środowiskowe. Niektóre z nich:

PWD	obecna ścieżkę pracy
HOME	ścieżka domowa użytkownika
USER	login użytkownika
PATH	zestaw ścieżek, na których Linux będzie szukał aplikacji
RANDOM	poda losową liczbę całkowitą z przedziału [0 .. 32767]
$	nr procesu aktualnej sesji

`readlink` : ekspandowanie nazwy pliku do pełnej lokalizacji.

```
$ readlink -f mojplik.txt
/home/login/mojplik.txt
```

Dodatek

Inne przydatne polecenia do manipulacji na nazwach plików

`$ dirname /home/login/mojplik.txt` `/home/login`	wyciągnie samą ścieżkę, wycinając nazwę
`$ basename /home/login/mojplik.txt` `mojplik.txt`	wyciągnie samą nazwę, wycinając ścieżki
`$ basename /home/login/mojplik.txt .txt` `mojplik`	i na dodatek usunie rozszerzenie `.txt`

Wynik takiej operacji możemy np. wstawić do zmiennej:

```
$ AnalysisDir=$(dirname /home/login/mojplik.txt)
```

⦿ Skrypt w Bashu: to plik z poleceniami, interpretowany przez Basha.

W pliku `hello.sh` napiszmy najprostszy skrypt Basha :

```
#!/bin/bash
echo "Hello, World!"
```

Pierwsza linia to tzw "shebang". Mówimy w niej, jaki język ma interpretować kod (tu: bash)
Pytanie, jak wywołać ten skrypt. Próba wprost (`./hello.sh`) się nie uda.
Aby zrozumieć powód, wylistujmy:

```
$ ls -og hello.sh
```

Jak widać, plikowi brak praw do wykonania.
Wkrótce je nadamy, ale na razie włączmy skrypt inaczej:

```
$ source hello.sh      lub:
$ . hello.sh
```

Na skutek takiego wywołania, kod będzie częścią naszej sesji (i procesu) .
Teraz nadajmy skryptowi prawa wywołania i następnie wywołajmy go jak każdą aplikację:

```
$ chmod 755 hello.sh
$ ./hello.sh
```

Linux otworzy dla niego podproces i w nim wywoła skrypt. Ten skrypt nie jest już częścią naszej sesji. Czyli wtedy domyślnie zmienne z naszej sesji nie przeniosą się do skryptu.
Zademonstrujmy to. Utwórzmy zmienną,

```
$ var=123
```

I zmieńmy skrypt na taki:

```
#!/bin/bash
echo _${var}_
```

Teraz wywołajmy skrypt na dwa odmienne sposoby:

```
$ source hello.sh          →          _123_
$ ./hello.sh               →          __
```

Oczywiście, poleceniem `export` można wymusić widoczność zmiennej w podprocesach:

```
$ export var
$ ./hello.sh               →          _123_
```

Skrypt c.d.: zmienne specjalne

Aby umożliwić wywołanie skryptu z opcjami wejścia, w kodzie dostępne są zmienne specjalne:

`$0`	nazwa bieżącego skryptu lub sesji
`$#`	ilość opcji, jaką wpisał użytkownik przy wywołaniu skryptu
`$1 $2 $ ...`	kolejne opcje wywołania
`$@`	lista opcji wywołania, oddzielonych spacją

Napiszmy skrypt `inputarg.sh` :

```
#!/bin/bash
echo $#
echo $0 $1 $2 $3
```

i wywołajmy go tak:

```
$ ./inputarg.sh -opt abc 123          →          3
                                                 -bash -opt abc 123
```

Pętle. Bash ma kilka rodzajów pętli: for, while, until, select.

Pętle są konieczne przy seryjnej obróbce danych. Również potrzebne przy posyłaniu zadań na farmę. Omówimy tu głównie pętlę for (są dwa rodzaje takiej pętli) i przez chwilę while.

- for - jako pętla zakresowa. Iteruje ona po liście: liczb/stringów, ale i plików/ścieżek. Typowa składnia:

```
for {zmienna} in {lista} ; do
  {działania}
done
```

- Przykłady różnych rodzajów list w nagłówkach „zakresowego for" :

```
for i in a1 b2 c3 ; do
for i in {03..12..3} ; do
for i in $(seq 3 3 12) ; do
for i in a*.txt ;  do          ←   po plikach w katalogu, zgodnych ze wzorcem
for i in */ ;  do              ←   po wszystkich podścieżkach katalogu
for i in $@ ;  do              ←   po opcjach wywołania skryptu
```

- Przykładowy skrypt z pętlą zakresową:

```
#!/bin/bash
for i in {01..05} ; do
  echo $RANDOM > myrandom_${i}.dat      ← każdemu z plików wpisze liczbę losową
done
```

Pętle c.d.

Inny przykład:

```
#!/bin/bash
for i in a*.txt ; do
  readlink -f $i
done
```

← każdy plik z listy zostanie wypisany z pełną ścieżką

Dodatek:

Właściwie, pętlę for można nawet napisać w 1 linii, w prompcie.
To się nazywa "one-liner".
Powyższy przykład w wariancie one-liner tworzy plik z listą nazw plików:

```
$ for i in a*.txt; do readlink -f $i ; done > myfiles.list
```

⚠ One-linery są poręczne, ale trudna czytelność może być dla nas pułapką.
A błąd przy pracy z plikami może nas sporo kosztować.
Dlatego stosujmy one-linery bardzo ostrożnie.

- Istnieje też pętla for – jako pętla „w stylu C" (C-style for loop). Przebiega ona po indeksie.
  Typowa składnia:

```
for (( i=3; i<8; i+=2 )) ; do
  echo $i
done
```

← nie poprzedzamy zmiennych znakiem $

- Aby umieć czytać plik linia-po-linii, zajrzyjmy na chwilę do pętli while :

```
while read SomeLine; do
  echo $SomeLine
done < plik_do_czytania.txt
```

Jest prawie dobrze, ale w czytanych liniach znikają wcięcia. Lepiej tak:

```
while IFS=read -r SomeLine; do
  echo "$SomeLine"
done < plik_do_czytania.txt
```

Dodatek: nawiasy (( )) służą arytmetyce na liczbach całkowitych. [tutaj]
Wyczerpujące omówienie wszystkich nawiasów: [tutaj]

⦿ Blok warunkowy if / elif / else. Jego schemat ogólny wygląda tak:

```
if {warunek logiczny} ; then
  {działania}
elif {warunek logiczny} ; then      ← opcjonalnie
  {działania}
else                                ← opcjonalnie
  {działania}
fi
```

Czym jest ten warunek logiczny i jak go kodujemy?
Warunek sprawdza relację pomiędzy liczbami czy stringami. Ale też status pliku czy zmiennej.
Dobre ściągawki z operatorami warunku są np. [tutaj] i [tutaj].

Zobaczmy kilka przykładów warunku logicznego, najpierw z operatorem `[ ]` :

```
[ 1 == 2 ]                          ← koniecznie wszędzie spacje
[ $var1 == $var2 ]
[ "$var1" == "1" ]                  ← bezpieczniej otoczyć obiekty cudzysłowami
[ ! 1 == 2 ]                        ← aby zanegować, poprzedzamy nawias znakiem !
[ ! "$var1" == "1" ]
```

- Operatory mniejszości/większości, które można wpisać do [ ], wyglądają nieefektownie:

```
[ 1 -lt 2 ]
[ 2 -gt 1 ]
```

→ Wymyślono więc lepszą wersję tych operatorów: `[[ warunek ]]` :

```
[[ 1 < 2 ]]
```

- Warunek logiczny może też dotyczyć statusu pliku/ścieżki czy zmiennej. Wypiszmy kilka:

```
[ -e {plik} ]                       ← czy plik istnieje?
[ -d {ścieżka} ]                    ← czy obiekt jest katalogiem?
[ -v var ]                          ← czy zmienna var istnieje?
[ -n "$var" ]                       ← czy zmienna var ma zawartość?
```

⊕ Dodatek:

W przypadku porównywania wyrażeń arytmetycznych (w tym w zmiennych) użyć można:

```
(( 1 < 2 ))
```

Blok warunkowy if c.d.

Przykładowy skrypt z pętlą for i warunkiem if (możesz np. nazwać go `forif.sh`):

```
#!/bin/bash
for i in {a..e} ; do
  for j in {1..5} ; do
    filename=${i}${j}.txt
    if [ ! -e $filename ] ; then
      echo Creating file $filename
      touch $filename
    fi
  done
done
```

Dodatek:

Prosty one-liner w prompcie z blokiem if: (nie nadużywajmy tej techniki z uwagi na ryzyko błędu)

```
$ if [ -e a1.txt ] ; then echo "This file exists" ; fi
```

Dodatek:

W bloku warunkowym można też użyć poleceń `break` i `continue`, tak jak w C/Python:

Przykładowy skrypt:

```
#!/bin/bash

for i in {01..10} ; do
  filename=myrandom_${i}.dat
  if ! [ -e $filename ] ; then
    echo "File $filename does not exist."
    break
  fi
  read val <$filename
  if (( ${val} < 15000 )) ; then
    continue
  fi
  echo "In File $filename val = $val ≥ 15000"
done
```

Z racji ograniczeń czasowych, tu kończymy naukę Basha.
Jeżeli chcesz nauczyć się o nim więcej, w następnych krokach rozważ zapoznanie się z:
- funkcjami (np. [tutaj] i [tutaj])
- tablicami (arrays; np. [tutaj] i [tutaj])

- Przykład skryptu z pętlą i warunkiem.

- W katalogu `Smash` mamy 5 plików z wynikami symulacji relatywistycznego zderzenia jąder atomowych. Nie zajmiemy się tu fizyką ani symulatorem. Istotne, że w każdej ze ścieżek `run1..5` jest ścieżka `outdir`, mieszcząca dane wyjściowe: `particle_lists.oscar`.

  Napiszmy w Bashu skrypt `checkfiles.sh`, który przebiegnie po 8 możliwych takich plikach (pierwszych 5 istnieje, dalszych 3 już nie). Skrypt ma zliczyć sukcesy i porażki – i wypisać wynik.

```
#!/bin/bash

N_present=0
N_missing=0

for i in {1..8} ; do
  filename=run${i}/outdir/particle_lists.oscar
  if [ -e $filename ] ; then
    (( N_present ++ ))
  else
    (( N_missing ++ ))
  fi
done

echo We found: $N_present present and $N_missing missing files.
```

- Nadaj prawa wywołania i wykonaj skrypt. Następnie uogólnimy go (do `checkfiles2.sh`), aby zakres katalogów podawał użytkownik. Również tu – przetestuj skrypt.

```
#!/bin/bash

N_present=0
N_missing=0

if [ ! $# == 2 ]; then
  echo Usage: ./checkfiles2.sh [run_from] [run_to]
  exit
fi

for i in $(seq $1 $2) ; do
  filename=run${i}/outdir/particle_lists.oscar
  if [ -e $filename ] ; then
    (( N_present ++ ))
  else
    (( N_missing ++ ))
  fi
done

echo We found: $N_present present and $N_missing missing files.
```